

nff on battery

Energy cost of the nff toolchain on ESP32, and the parameter set for a
BLE-only 6 000 mA h / 12-month sensor node

Power budget — Revision A

Target: ESP32-WROOM-32E, BLE-only, one gateway per site

2026-07-13

† **Every current, duration and energy figure in this document is a placeholder.** None has been measured. They are internally consistent and drawn from the ESP32 datasheet’s order of magnitude, and they are good enough to decide *feasibility* — which is what §1 does — but they are not good enough to sign a contract against. The rig that will replace them is specified in `datasheet.tex`, and §8 explains why that rig **cannot measure the most important number in this document** (the sleep floor), and what instrument can.

Abstract

This document does three things. It gives the energy cost of every **nff** operation that runs on an ESP32 (§2, mock values). It gives the **nff** SDK settings that matter under a power constraint, and states plainly the two things the SDK cannot currently do at all (§3). And it gives the ESP32/BLE parameter set for the stated duty cycle (§4–§5).

The headline is a distinction, not a number. The requirement says the device “wakes for approximately 30 s every 5 min”. *Awake for 30 s* and *the CPU runs for 30 s* are not the same requirement, and the feasibility of this product turns entirely on which one gets built. The literal reading — wake, advertise in a busy loop for 30 s, sleep — yields **6 weeks** on a 6 000 mA h cell, and **20 days** if it is built on a devkit board. The target is 12 months. That gap is not closable by tuning the radio, because the radio is not where the energy goes.

Two designs do close, with roughly $1.5\times$ margin, and both are given in full (§1). One shortens the CPU-active window to 2.5 s. The other keeps the literal 30 s window but spends 29 s of it in automatic light sleep, with the BT controller holding the advertising timing. They land within 5 % of each other — which is itself the most useful result here: once the CPU is asleep, the **boot sequence, not the radio and not the wake duration, becomes the largest single line item in the budget.**

Contents

1	Verdict	3
1.1	The budget	3
1.2	Four designs	3
1.3	The closure equation	3
1.4	Why the verdict survives the mock values	4
1.5	The one measurement that converts this document — TAKEN 2026-07-14	4
1.6	Cycle profile (D2, recommended)	5
1.7	Cycle energy, phase by phase (D2)	5
1.8	D3 — the literal 30-second window, for comparison	6
2	Energy cost of each nff operation	6
2.1	Steady-state and command operations	7
2.2	OTA — the dominant single operation	7
2.3	Why WiFi was never an option	7
3	Where the nff SDK actually stands	8
3.1	Two structural gaps	8
3.2	Settings that do apply	10
3.3	Two hardcoded behaviours that must become configurable	11
3.4	Required SDK work	13
4	ESP32 and BLE parameters	14
4.1	What the WROOM-32E can and cannot do	14
4.2	Recommended configuration	16
4.3	Budget the exception cycles	17
5	The 31-byte advertisement	17
6	The sleep floor is a board property, not a chip property	18
7	Battery: mA h is not the binding constraint	19
8	How this budget can lie to you	19
9	Verification sequence	20
10	Measurement log	21

1 Verdict

1.1 The budget

Quantity	Derivation	Value
Battery, nominal	Given	6 000 mA h
Battery, <i>usable</i>	85 % derate: self-discharge, low-temperature capacity loss, cut-off voltage headroom (§7)	5 100 mA h
Target life	Given: 12 months	8 760 h
Mean current budget	5 100 mA h / 8 760 h	582 μA
Cycles per year	8 760 h $\times$ 3 600 s/h / 300 s	105 120
Charge budget per cycle	582 μ A $\times$ 300 s	48.5 μA h

Everything the device does in a 5 min cycle — boot, initialise the BT controller, read the sensor, advertise, tear down, and sleep for the remaining 4.5 min — must fit inside 48.5 μ A h. That is the whole problem, stated once.

1.2 Four designs

	Board	CPU-active	Sleep floor	I_{mean}	Life
D0	Devkit	31.2 s	6.5 mA	10.8 mA	20 days
D1	Custom	31.2 s	20 μ A	5.01 mA	6.1 weeks
D2	Custom	2.5 s	20 μ A	380 μ A	18.4 months
D3	Custom	1.2 s	20 μ A	394 μ A	17.7 months
<i>Budget — mean current must not exceed</i>				582 μA	<i>12 months</i>

D3's 1.2 s of CPU-active time sits inside a 30 s advertising window; the CPU is in light sleep for the other 29 s (§1.6).

D0 — the literal spec, built the obvious way, on a devkit.

FAIL by 18 $\times$. Note that D0 fails *on its sleep floor alone*: 6.5 mA of AMS1117 quiescent current, CP2102 bridge and power LED, drawn for 4.5 min of every cycle, is already 11 $\times$ the entire budget before the firmware executes one instruction (§6).

D1 — the literal spec, on a proper board.

FAIL by 8.6 $\times$. The board is fixed; the firmware is not. 30 s of CPU-active time at roughly 48 mA is 400 μ A h per cycle against a 48.5 μ A h budget.

D2 — short window. *Recommended.*

PASS, margin 1.53 $\times$. Identical behaviour from the gateway's point of view; the wake window is 2.5 s instead of 30 s. No extra parts.

D3 — literal 30 s window, held in light sleep.

PASS, margin 1.48 $\times$. Preserves the stated 30 s advertising window verbatim; the CPU only *runs* for 1.2 s of it. Costs a 32.768 kHz crystal on the BOM.

1.3 The closure equation

For a period T , a sleep floor I_{sleep} and a mean CPU-active current I_{awake} , the longest CPU-active window that still meets a mean-current budget I_{budget} is

$$t_{\text{on,max}} = T \cdot \frac{I_{\text{budget}} - I_{\text{sleep}}}{I_{\text{awake}} - I_{\text{sleep}}} \implies 300 \text{ s} \cdot \frac{582 \mu\text{A} - 20 \mu\text{A}}{43.2 \text{ mA} - 20 \mu\text{A}} = 3.9 \text{ s}.$$

3.9 s is the hard ceiling on CPU-active time per 5 min cycle. Every design decision in this document is downstream of that one number. Spending the entire 6 000 mA h with no derate at all buys 4.6 s — so the ceiling is robust: it does not hinge on how generous the battery derating is.

Note what the equation is *insensitive* to. Halving I_{awake} from 43 mA to 21 mA — an aggressive, expensive, silicon-level win — only doubles $t_{\text{on,max}}$, to 7.8 s. That is still nowhere near 30 s. **No achievable reduction in current closes a 30 s CPU-active window.** Only reducing the *time* does. This is why the recommendation is a firmware-architecture change and not a component selection.

1.4 Why the verdict survives the mock values

Every current in this document is invented (§8, L1), which invites the obvious objection: *if the inputs are made up, why believe the conclusion?* Because the conclusion does not use them. The failure of D1 can be shown with no number from this document at all.

Grant the design every advantage:

- **No battery derate.** Spend the entire 6 000 mA h, not the 5 100 mA h of §1.1. Budget becomes 685 μA .
- **A free sleep.** Set $I_{\text{sleep}} = 0$. The device costs nothing for 4.5 min of every cycle.
- **A silent radio.** No advertising, no TX. The CPU merely *runs*.

The duty cycle is fixed by the requirement at 30 s/300 s = 10 %. So the maximum current the device may draw while awake is

$$I_{\text{awake,max}} = \frac{685 \mu\text{A}}{0.10} = 6.85 \text{ mA}.$$

There is no operating mode in which an ESP32 executes code at 6.85 mA. Espressif’s datasheet puts the *lowest* CPU-executing state — modem sleep, 80 MHz, radio off — at roughly 20 to 31 mA. The requirement is therefore over budget by at least 3× **with a perfect battery, a free sleep, and the radio switched off**, before the sensor is read or a single advertisement is sent.

The mock values decide whether the answer is “6 weeks” or “9 weeks”. **They do not decide whether it fails.** That conclusion rests on three numbers, none of them ours: the customer’s 6 000 mA h, the customer’s 10 % duty cycle, and Espressif’s datasheet.

What the mock values *do* decide is the margin on D2 and D3 — whether they land at 18 months, or at 11, or at 30. Those figures are not yet trustworthy, and §1.5 gives the single measurement that makes them so.

1.5 The one measurement that converts this document — **TAKEN 2026-07-14**

The verdict hinges on exactly one quantity: I_{awake} , the mean current with the CPU running and BLE advertising. Everything else is either arithmetic or a design target.

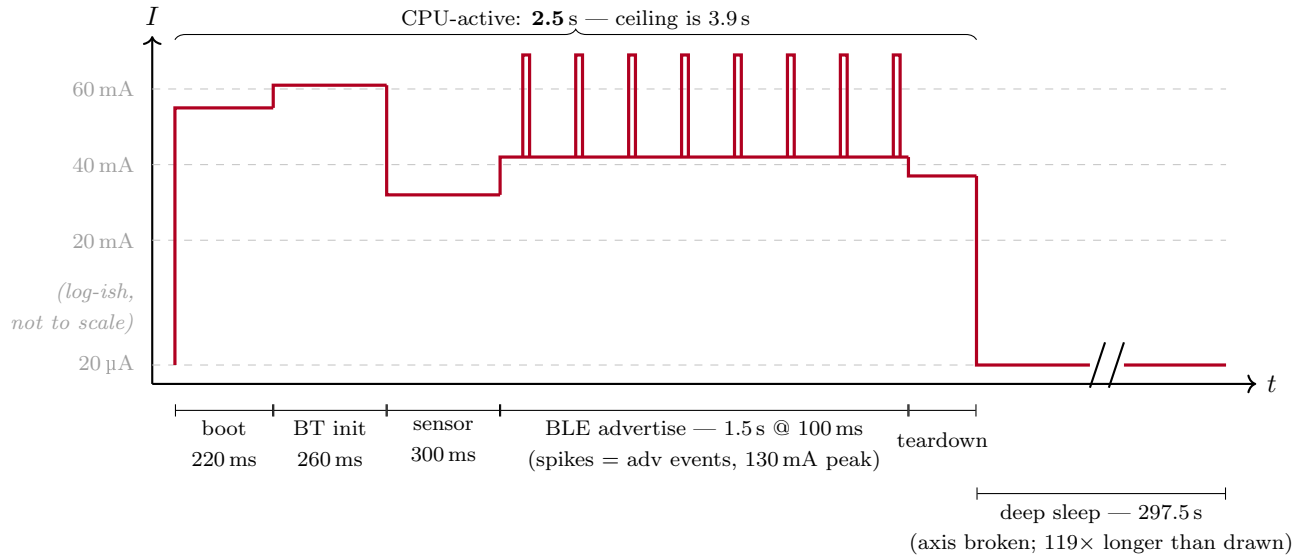
Done — for WiFi. The rig ran on 2026-07-14 with the `esp32-nffload` workload. De-embedded to the module’s 3.3 V rail: $I_{\text{core}} = 35.0 \text{ mA}$ (model said 35 mA — **exact**); WiFi connected-idle 49.7 mA; a 1 MiB OTA 150.5 mA for 8.6 s. **The current model was right; the durations were not** — OTA throughput is 122 kB/s, not the 29 kB/s assumed, so k_t moves from 1.93 to 8.1. **I_{awake} for BLE remains unmeasured** — the SDK has no BLE, so there is nothing to flash. **The sleep floors remain unmeasured** and cannot be measured here (L2).

The existing shunt rig can measure it. I_{awake} is tens of mA — comfortably inside the rig’s range, unlike the sleep floor, which is not (§8, L2). The experiment is small:

1. Flash a bare BLE advertiser — no nff, no WiFi, no sensor — onto the ESP32 devkit.
2. Put it on the rig per `datasheet.tex` (DUT USB *not* connected).
3. `nff power measure --for 30`. That number is I_{awake} .

That single figure replaces the largest mock in the document and turns the D2/D3 margins from plausible into measured. It cannot validate the sleep floor — nothing on this bench can (§8, L2) — but the sleep floor is 5 % of the budget, and I_{awake} is 95 % of it.

1.6 Cycle profile (D2, recommended)



1.7 Cycle energy, phase by phase (D2)

Phase	Duration	I_{mean} †	Charge	% cycle
ROM boot + 2nd-stage bootloader (validate skipped)	220 ms	55 mA	12.1 mA s	10.6 %
App init + NimBLE controller and host init	260 ms	62 mA	16.1 mA s	14.1 %
Sensor read (I ² C) + pack + CMAC	300 ms	30 mA	9.0 mA s	7.9 %
BLE non-connectable advertise, 100 ms interval, 9 dBm	1 500 ms	42 mA	63.0 mA s	55.3 %
BT teardown + deep-sleep entry	220 ms	35 mA	7.7 mA s	6.8 %
CPU-active subtotal	2.50 s	43.2 mA	107.9 mA s	94.8 %
Deep sleep	297.5 s	20 µA	5.95 mA s	5.2 %
Cycle total	300 s	380 µA	31.6 µA h	
<i>Annual charge</i>		31.6 µA h × 105 120 = 3 325 mA h		
<i>Life on 5 100 mA h usable</i>		18.4 months — margin 1.53×		

Read the last column, not the first. Deep sleep is 99.2 % of the elapsed time and 5.2 % of the energy. The largest single line item is the advertising hold; the second and third are *boot and initialisation* — 24.7 % of the cycle budget spent before the device has done anything at all. In D3, where the advertising hold is light-slept away, boot and init become the **largest** item outright. Optimisation effort belongs there, not on the radio.

1.8 D3 — the literal 30-second window, for comparison

Phase	Duration	$I_{\text{mean}} \uparrow$	Charge	CPU?
Boot + bootloader + 32.768 kHz crystal settle	420 ms	55 mA	23.1 mA s	yes
App init + NimBLE init	260 ms	62 mA	16.1 mA s	yes
Sensor read + pack	300 ms	30 mA	9.0 mA s	yes
Advertising hold — 500 ms interval, automatic light sleep between events	30.0 s	1.9 mA	57.0 mA s	no
Teardown + deep-sleep entry	220 ms	35 mA	7.7 mA s	yes
Awake subtotal	31.2 s	3.6 mA	112.9 mA s	
Deep sleep	268.8 s	20 μ A	5.4 mA s	
Cycle total	300 s	394 μA	32.9 μA h	
<i>Life on 5 100 mA h usable</i>		17.7 months — margin 1.48×		

The 30 s advertising hold costs 57 mA s; D2’s 1.5 s hold costs 63 mA s. **They cost the same.** A 20× longer window at 22× lower current is a wash. **So the choice between D2 and D3 is not a power decision at all** — decide it on these grounds instead:

	D2 — short window	D3 — literal 30 s
BOM	No extra parts.	Requires a 32.768 kHz crystal for the BT low-power clock.
Firmware	Simple. No <code>esp_pm</code> , no tickless idle, no BT modem sleep.	Needs <code>CONFIG_PM_ENABLE</code> , tickless idle and BT modem sleep, all interacting. The hardest class of bug to find.
Gateway	1.5 s to catch 15 adv events. Needs a <i>continuously</i> scanning gateway.	30 s window, 60 adv events. Tolerant of a gateway that scans intermittently, or round-robins across many nodes.
Reception margin	Lower. A gateway busy elsewhere for 1.5 s misses the cycle entirely.	High. This is D3’s real justification.

Choose **D2** if the gateway scans continuously and serves few nodes. Choose **D3** if one gateway must round-robin across many nodes, or if the 30 s window is contractual. *Do not choose between them on battery life — there is nothing to choose.*

The two paths have different BOMs. Decide before the board is laid out. D2 wants *no* 32.768 kHz crystal — fitting one costs roughly 200 ms of startup on *every* wake, which is 10 % of D2’s entire CPU-active budget, to buy a timing accuracy the design does not need. D3 cannot work without one. Retrofitting the crystal later is a board spin; omitting it later is not.

2 Energy cost of each nff operation

Mock values $\uparrow$. Energy is reported at the module’s 3.3 V rail ($E = 3.3 \text{ V} \cdot I \cdot t$), which is the quantity a battery design cares about. The rig in `datasheet.tex` measures the whole devkit at its 5 V input and must be de-embedded before its figures can be substituted here.

2.1 Steady-state and command operations

Operation	Duration	I_{mean}	Energy	Radio
<code>nff_init()</code> — NVS read, panic hook, OTA boot-check	12 ms	40 mA	1.6 mJ	—
WiFi associate + DHCP *	3.1 s	64.7 mA	662 mJ	TX/RX
SNTP time sync (<i>cold boot; hardcoded</i>)	1.8 s	95 mA	564 mJ	TX/RX
<code>nff_connect()</code> — mTLS handshake, MQTT CONNECT, SUBSCRIBE	1.6 s	110 mA	581 mJ	TX/RX
Heartbeat publish (QoS 1 retained, 250-byte JSON)	45 ms	120 mA	17.8 mJ	TX
MQTT keepalive PINGREQ	12 ms	115 mA	4.6 mJ	TX/RX
<code>nff_loop()</code> idle tick (connected, no work)	0.4 ms	38 mA	0.05 mJ	—
<code>nff_log()</code> one line (UART + RTC ring)	0.3 ms	42 mA	0.04 mJ	—
Inbound command: ECDSA-P256 verify (<i>CPU-bound</i>)	55 ms	62 mA	11.3 mJ	—
↪ ping response	18 ms	118 mA	7.0 mJ	TX
↪ diag response (heap + RSSI)	30 ms	118 mA	11.7 mJ	TX
↪ reboot (ack + restart)	0.9 s	88 mA	261 mJ	TX
Crash report publish (2.5 kB retained) + NVS erase	180 ms	105 mA	62 mJ	TX
Reconnect attempt — one failed TLS handshake	1.9 s	108 mA	677 mJ	TX/RX
Bootstrap announce + cert rollover (chunked NVS writes)	3.2 s	112 mA	1.18 J	TX/RX

2.2 OTA — the dominant single operation

OTA phase (1.2 MB image)	Duration	I_{mean}	Energy	Charge
Command verify (ECDSA, nonce ring, anti-downgrade)	55 ms	62 mA	11.3 mJ	—
<code>ota_started</code> publish	40 ms	120 mA	15.8 mJ	—
HTTP GET stream + flash write (4 kB chunks) *	9.8 s	150.5 mA	4.89 J	0.41 mA h
NVS commit (<code>ota_trial</code>) + reboot	0.7 s	90 mA	208 mJ	—
OTA download total *	9.9 s	150 mA	4.92 J	0.41 mA h
Probation soak (NFF_OTA_MIN_HEALTHY_MS = 5 000 ms)	5.0 s	82 mA	1.35 J	0.11 mA h
Rollback (partition switch, reboot, <code>ota_result</code>)	0.9 s	88 mA	261 mJ	—

An OTA costs 0.41 mA h — 0.007 % of the battery *. **OTA is not the problem.** A device could take an update every week for its entire life and spend under 1 % of its energy on it. It is the *recurring* costs that kill it.

2.3 Why WiFi was never an option

Recurring cost, as the nff SDK ships	Per event	Events/yr	Annual charge
Heartbeat, <code>heartbeat_interval_s</code> = 30 (default)	1.5 μ A h	1 051 200	1 577 mA h
Heartbeat, <code>heartbeat_interval_s</code> = 300 (Kconfig max)	1.5 μ A h	105 120	158 mA h
WiFi associated + idle (<i>the SDK never enables WiFi power save</i>)	—	continuous	$\approx$876 000 mA h

An nff device on WiFi, exactly as the SDK ships, lasts 2.5 days on this battery. The default heartbeat alone spends 26 % of a 6 000 mA h cell per year — and that is only the *publishes*. The association that carries them holds the radio at roughly 100 mA continuously, because `nff-sdk-c` never calls `esp_wifi_set_ps()`. Even with WiFi power save and DTIM 3 — which would have to be added — an always-associated station averages roughly 7 mA, giving **36 days**. *The BLE-only decision in the requirement is not a preference. It is the only transport that can meet this budget, and it was the right call.*

3 Where the nff SDK actually stands

3.1 Two structural gaps

nff-sdk-c cannot run on this device today. Not “needs tuning” — cannot run. Two gaps, both established by reading the source rather than inferred:

1. There is no BLE. `include/nff_port.h` is the SDK’s *only* hardware contract (“all nine SDK modules call only `nff_port_*` functions”). Its transport entries are `nff_port_mqtt_*` (MQTT over mTLS) and `nff_port_https_get_stream` (HTTPS, for OTA). There is no `nff_port_ble_*`. A repository-wide search for BLE, NimBLE, bluedroid, `esp_bt_*`, `esp_ble_*`, `gatt` and `advertis*` returns **zero hits in any firmware source**. (`nff_crash.c`’s `bt_json` is a *backtrace*; `nff-pentester/.../ble.py` is a host-side Python tool.)

2. There is no sleep. No `esp_deep_sleep`, `esp_light_sleep`, `esp_sleep_enable_*`, `esp_pm_*`, `WIFI_PS_*` or `rtc_gpio_*` anywhere in `nff-sdk-c/`. `nff_loop()` presumes a continuously-associated station, and the fleet marks a device offline after $2 \times \text{heartbeat_interval_s}$ of silence — i.e. the protocol is *built on* the assumption that the device is always reachable. The SDK’s entire low-power story is one sentence in `docs/PROTOCOL.md`: `raise heartbeat_interval_s`.

3.2 Settings that do apply

Symbol	Default → set	Why
cfg.heartbeat_interval_sCONFIG_NFF_HEARTBEAT_INTERVAL_S	30 → 300	The only shipped power knob, and 300 is the Kconfig ceiling. It aligns the heartbeat to the 5 min wake cycle: one heartbeat per wake, and none in between.
NFF_MQTT_BUFFER_SIZE	1024 → 1024	Not a power knob — a land-mine. The OTA command is roughly 420 bytes; Pub-Sub-Client's 256-

3.3 Two hardcoded behaviours that must become configurable

The reconnect backoff ladder is the SDK's worst battery failure mode, and it is not a setting. `nff_core.c:31` hardcodes `{1000, 2000, 4000, 8000, 16000, 32000, 60000}` ms, *saturating at 60s*. A device that cannot reach its peer therefore retries forever, once a minute, and each retry is a full radio bring-up plus a TLS handshake — about 677 mJ, or 57 μ A h. That is 500 mA h per month, **with no telemetry to show for it**, while the fleet sees only a device that has gone dark. It must be capped, and it must be able to give up and go back to sleep.

SNTP is unconditional, and it blocks `nff_connect()`. `nff_port_esp32_arduino.c:60` hardcodes `configTime(0, 0, "pool.ntp.org", ...)` and polls for up to roughly 20 s on a cold boot with an unset RTC — mTLS is *skipped* until it lands. A duty-cycled device cold-boots *every cycle*. As written, this would spend 20 s on NTP per wake: **five times the entire CPU-active budget**, on a service the device does not need.

3.4 Required SDK work

Addition	Notes
nff_port_ble_adv_start / _stop / _set_payload / _set_tx_power	New entries in nff_port.h. The good news: this is an addition, not a rewrite. The PAL is already the sole hardware contract, so a BLE port slots in beside the MQTT one without touching the nine core modules.
nff_port_sleep_deep(ms) / nff_port_sleep_light(ms)	New. Nothing in the PAL touches power today.
nff_run_once() — do work, return	Beside nff_loop() / nff_start_task(). The latter pins an 8kB, priority-2 FreeRTOS task to Core0 that <i>never exits</i> — structurally the wrong shape for a device whose whole job is to reach deep sleep.
State retained across deep sleep	The mechanism already exists. nff_crash.c keeps its log ring in RTC_NOINIT. Reuse it for seq, backoff state and OTA trial state, so that a cold boot every cycle is not a <i>cold</i> boot.
Packed 27-byte heartbeat codec	The heartbeat is a 250-byte JSON (char payload[416], nff_heartbeat.c:38). A BLE 4.2 legacy advertisement carries 31 bytes <i>total</i> . It does not fit, and it cannot be made to fit — see §5.
Gateway-side MQTT bridge	The gateway holds the mTLS identity and republishes to nff/{project}/devices/{id}/heartbeat. The device’s ECDSA-verified command path (nff_cmd.c) must move to a BLE GATT write, or be dropped.
OTA over BLE	nff_port_https_get_stream has no BLE analogue. 1.2 MB over BLE 4.2 1M PHY (there is no LE 2M PHY on this part — §4) at a realistic 15 kB/s is about 80 s of connected

4 ESP32 and BLE parameters

4.1 What the WROOM-32E can and cannot do

	Capability	Consequence
yes	Bluetooth 4.2, BR/EDR + BLE	BLE works. BR/EDR is dead weight — compile it out.
yes	Deep sleep, RTC-timer wake, $\approx 10\ \mu\text{A}$ (module)	Covers the 4.5 min of every cycle.
yes	Light sleep, $\approx 0.8\ \text{mA}$	This is what makes D3 possible.
no	Extended advertising	You get 31 bytes of advertising payload, not 255. Extended advertising is a BT 5.0 feature. If any part of the payload design assumed otherwise, it breaks here.
no	LE 2M PHY	All BLE traffic, including any future OTA, runs at 1M.
no	LE Coded PHY (long range)	Range must be bought with TX power, not with coding gain.

If one part can still be changed, change this one. An ESP32-C3-MINI-1 or ESP32-C6 is the same price class and footprint class, and gives BLE 5.0 — extended advertising (255-byte payloads, so the nff heartbeat would fit *as-is*), LE 2M PHY, and Coded PHY for range — at roughly *half* the active current. The WROOM-32E’s dual Xtensa cores and BR/EDR radio are pure liability in a device whose entire job is to wake, read one sensor, and go back to sleep. *The design closes on the WROOM-32E anyway* — at $1.5\times$ margin instead of roughly $3\times$. This is a recommendation, not a blocker.

4.2 Recommended configuration

Setting	D2	D3
CONFIG_BT_NIMBLE_ENABLED	y	

CONFIG_BT_CLASSIC_ENABLED

n

Spend the link margin. Set TX power to 9 dBm, the maximum. This is counterintuitive, and it is correct. The radio is on for roughly 2.5 ms of each 100 ms advertising interval — a 2.5 % duty cycle, inside a phase that is itself 0.5 % of the wall-clock cycle. Raising TX power from 0 dBm to 9 dBm moves the *cycle* budget by well under 1 %. A *missed* advertisement, by contrast, costs a whole wasted cycle — about 30 μ A h, or 60 % of a cycle’s entire budget, plus the lost reading. **The arithmetic says buy range with power and never think about it again.** The instinct to turn the radio down to save battery is, on this design, exactly backwards.

4.3 Budget the exception cycles

Three cycles are not like the others, and a design that budgets only the nominal cycle will drift.

Exception	Cost	Budget	Notes
OTA probation soak	0.11 mA h	12/yr	NFF_OTA_MIN_HEALTHY_MS = 5 s of <i>continuous</i> healthy soak exceeds the 3.9 s CPU-active ceiling on its own. Accept it as a long-wake exception; do not shorten the soak to make it fit.
OTA download	1.89 mA h	12/yr	23 mA h per year — 0.4 % of the battery. Negligible.
Crash report	0.02 mA h	rare	Negligible.
Unreachable gateway	57 μ A h per retry	unbounded	The only exception that matters. Per §3, the backoff ladder saturates at one full radio bring-up per minute, forever. Cap it, or a single dead gateway drains a 6 000 mA h node in weeks.

5 The 31-byte advertisement

BLE 4.2 legacy advertising carries 31 bytes of AdvData. One Manufacturer Specific Data structure costs 4 bytes of framing (length, type 0xFF, and a 2-byte company identifier), leaving **27 bytes** of payload.

Field	Bytes	Notes
<i>(AD framing: length, 0xFF, company ID)</i>	4	<i>Not payload.</i>
ver / packet type	1	Lets the codec evolve without a fleet-wide flag day.
seq	2	Wraps. The gateway deduplicates on (address, seq) — 15 adv events per wake are 15 copies of <i>one</i> reading, not 15 readings.
flags	1	Status, OTA trial state, low battery.
fw_build_id	2	Enough for the fleet to see what is actually deployed.
battery	1	Percent.
sensor payload	16	What is left — and it is the entire point of the device.
tag	4	AES-CMAC-128, truncated. Prevents a spoofed reading; does <i>not</i> give confidentiality.
Total	27	= the payload ceiling. Exactly full.

The device identity is free — do not pay for it twice.

The 6-byte static random BLE address is already in the advertising packet header. Spending payload bytes on a device ID would take a quarter of the sensor field to carry information the gateway already has.

The nff heartbeat does not fit, by 8×.

It is a 250-byte JSON (`char payload[416]`, `nff_heartbeat.c:38`). No compression story closes an 8× gap. The gateway must *reconstruct* the JSON heartbeat from the packed 27-byte record and publish it to the fleet — which is also where the mTLS identity and the SNTP-correct timestamp come from.

The Flags AD structure will silently eat 3 of your 27 bytes if you let it. Flags is only *required* for a discoverable device; a non-connectable beacon may omit it. NimBLE emits exactly the AD structures you hand it. Bluedroid's `esp_ble_gap_config_adv_data()` helper **inserts Flags for you** — use `esp_ble_gap_config_adv_data_raw()` instead. Get this wrong and the sensor field silently shrinks from 16 bytes to 13, which is the kind of thing that gets discovered after the enclosure tooling is cut.

6 The sleep floor is a board property, not a chip property

	Custom board	Devkit
ESP32-WROOM-32E, deep sleep, RTC timer only	10 μ A	10 μ A
LDO quiescent	1.6 μ A (MCP1700 class)	5 000 μ A (AMS1117)
USB–UART bridge	— (not fitted)	100 μ A (CP2102)
Power LED	— (not fitted)	1 400 μ A
Sensor(s), standby	3 μ A	3 μ A
Leakage, pull-ups, RTC_SLOW retention	5 μ A	5 μ A
Total sleep floor	20 μA	6.5 mA
Sleep alone, vs. the 582 μA budget	3.4 %	11× over

A devkit fails the requirement while doing nothing at all. Its 5 mA AMS1117 and its power LED draw more, asleep, than the entire per-cycle budget. No firmware can recover this. **Any battery-life number measured on a devkit is meaningless.**

And this is precisely the trap the shunt rig’s own datasheet describes — in reverse. That rig deliberately measures the *whole devkit board*, LDO and bridge and LED included, because for *its* purpose (the marginal energy of an OTA) those terms are constant and cancel. *Here they do not cancel. Here they are the answer.*

7 Battery: mA h is not the binding constraint

The 85 % derate used in §1 is not a fudge factor. It is a stand-in for four separate effects, and each must be checked against the actual cell.

Effect	What to check
Self-discharge	Chemistry-dependent, and <i>not</i> small over 12 months. Li-SOCl ₂ : about 1 % per year. Li-ion: 2 to 3 % <i>per month</i> — which alone would consume 24 to 36 % of the pack over the target life. The chemistry choice is a 30 % swing in the budget.
Low temperature	Capacity falls with temperature, and the sites are presumably not climate-controlled. Derate against the coldest expected ambient, not the datasheet’s 25 °C.
Cut-off voltage	The ESP32’s radio needs a solid ≥ 3.0 V. From a 3.6 V Li-SOCl ₂ cell, an LDO has almost no headroom by end of life; a buck-boost may be required — and then its own efficiency and quiescent current enter the budget.
Pulse capability	See below. This is the one that bites.

The battery’s ability to source 130 mA pulses matters more than its mA h rating. Li-SOCl₂ cells — the obvious chemistry for a 12-month, 6 000 mA h, low-average-current design — have high internal impedance and *passivate* during long idle periods. The consequences are specific, and they all land on this design:

- The roughly 250 mA boot inrush and the roughly 130 mA BLE TX pulse will sag the terminal voltage.
- **The worst pulse is the first one after a long idle** — which is exactly the pulse this design takes, every cycle, forever. A brownout there is a boot loop, and a boot loop on this battery is a dead device within days.
- A design that passes on the bench with a fresh cell can still fail in the field at month nine, as internal impedance rises with depth of discharge.

Mitigation: a hybrid layer capacitor (HLC), or a supercapacitor with a series limiting resistor across the cell, sized for the boot inrush. **Verification:** scope the rail during the *first* advertising event after an hour of deep sleep, on a *partially-discharged* cell. Not a fresh one — and not a bench supply, which will pass this test every time and tell you nothing.

8 How this budget can lie to you

In the spirit of the rig’s own honesty section: each of these is a way to arrive at a confident, plausible, *wrong* answer.

L1. Every number in this document is a mock. † Nothing here is measured. The arithmetic is sound, and the *ratios* are the point — a 27× spread between D0 and D2 does not collapse to 1× if the currents turn out to be 30 % off. But do not quote an absolute month figure to a customer from this revision.

- L2. The existing rig cannot measure the most important number here.** Its own datasheet says so: deep sleep at $10\mu\text{A}$ across a 1Ω shunt is $10\mu\text{V}$, roughly $100\times$ below the breadboard noise floor, and its capability envelope marks deep sleep **no**. **Every sleep-current figure in this document is unmeasurable on the shunt rig.** A $20\mu\text{A}$ floor needs an INA228 or a Nordic PPK2. *Do not accept a sleep figure from the shunt rig, and do not read its silence as a zero* — a 6.5mA devkit floor and a $20\mu\text{A}$ custom-board floor are the difference between 20 days and 18 months, and the shunt rig returns the same indistinguishable noise for both.
- L3. An average hides a brownout.** A design can meet its mean-current budget perfectly and still fail in the field, because the failure is a 130mA pulse into a passivated cell (§7), not a mA h overspend. The budget in §1 cannot see this, and neither can any integrating meter.
- L4. Bench conditions hide the recurring costs.** A device on the bench sits next to its gateway and never retries. A device in the field, behind a wall, retries — and the nff reconnect ladder saturates at one full radio bring-up per minute, forever (§3). **The 12-month figure assumes the gateway is always reachable.** It is a best case, and the SDK currently has no bounded-retry behaviour to make the worst case survivable.
- L5. “Awake” is not “CPU running”.** The requirement’s 30 s is ambiguous between the two, and the two differ by $12\times$ in battery life. If this document achieves one thing, let it be that nobody builds D1 by accident.

9 Verification sequence

Run in order. Each step gates the next. Steps 1–2 require a PPK2 or an INA228 — *not* the shunt rig (§8, L2).

- Sleep floor, on the board you will actually ship.** Deep sleep, radio down, sensor in standby. Must read $\leq 25\mu\text{A}$. *If it does not, nothing else counts* — and if it reads in mA , you are measuring a devkit (§6).
- CPU-active window.** Trigger one wake cycle and integrate. Must be $\leq 3.9\text{s}$ of CPU-active time and $\leq 48.5\mu\text{A h}$ of charge. This is the entire requirement, in one measurement.
- Boot cost.** Integrate from reset to the first instruction of `app_main`. Confirm `CONFIG_BOOTLOADER_SKIP_VALID` actually took effect — it is worth 2 to 3 % of the budget and it is easy to leave off.
- Reception, at range, at the real site.** 100 cycles; count gateway receipts. A missed cycle costs 60 % of a cycle’s budget in wasted advertising and, worse, loses the reading. *This is what the 9 dBm TX power is for.*
- Pulse integrity on a discharged cell.** Scope the rail during the first advertising event after 1 h of deep sleep, on a partially-discharged cell (§7). Not a bench supply.
- Unreachable-gateway drain.** Power the node with *no gateway present* and measure for 1 h; extrapolate. This is the failure mode the SDK does not bound, and it is the one that will be discovered in the field rather than on the bench.

Reporting. State the measured mean current and the CPU-active window plainly, and state *which instrument* produced the sleep figure. A battery-life number derived from a shunt-rig sleep measurement is not a number — it is the noise floor with a unit attached.

10 Measurement log

Date	Board	I_{sleep}	t_{on}	I_{mean}	Instrument / notes
2026-07-14	Devkit	not measured	—	49.7 mA	Shunt rig, $1\,\Omega$ uncalibrated . WiFi connected-idle, module rail.
2026-07-14	Devkit	not measured	8.6 s	150.5 mA	1 MiB OTA (HTTP, no TLS). 3.06×10^6 samples, $\text{ovr} = 0$.
2026-07-14	Devkit	—	—	35.0 mA	I_{core} : radio off, CPU 80 MHz. Matches the model exactly.

Revision A.1 — the WiFi column is now measured (, 2026-07-14; see `nff-power-characteristics.tex` §1.5). **Every BLE figure and every sleep floor remains modelled.** The nff SDK has neither a BLE transport nor any sleep support as of this revision (§3); the parameter sets in §4–§5 therefore specify work to be done, not settings to be changed.*